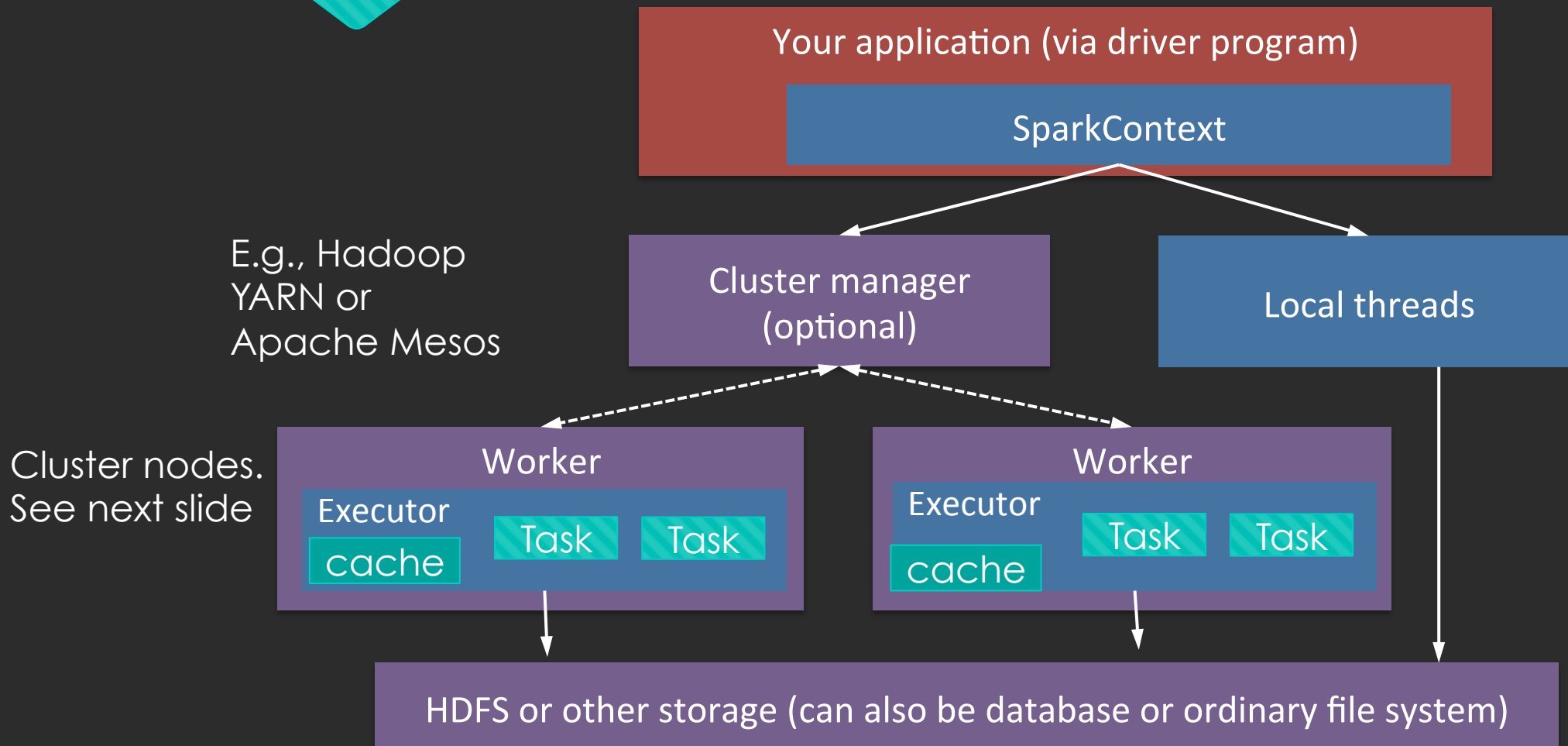
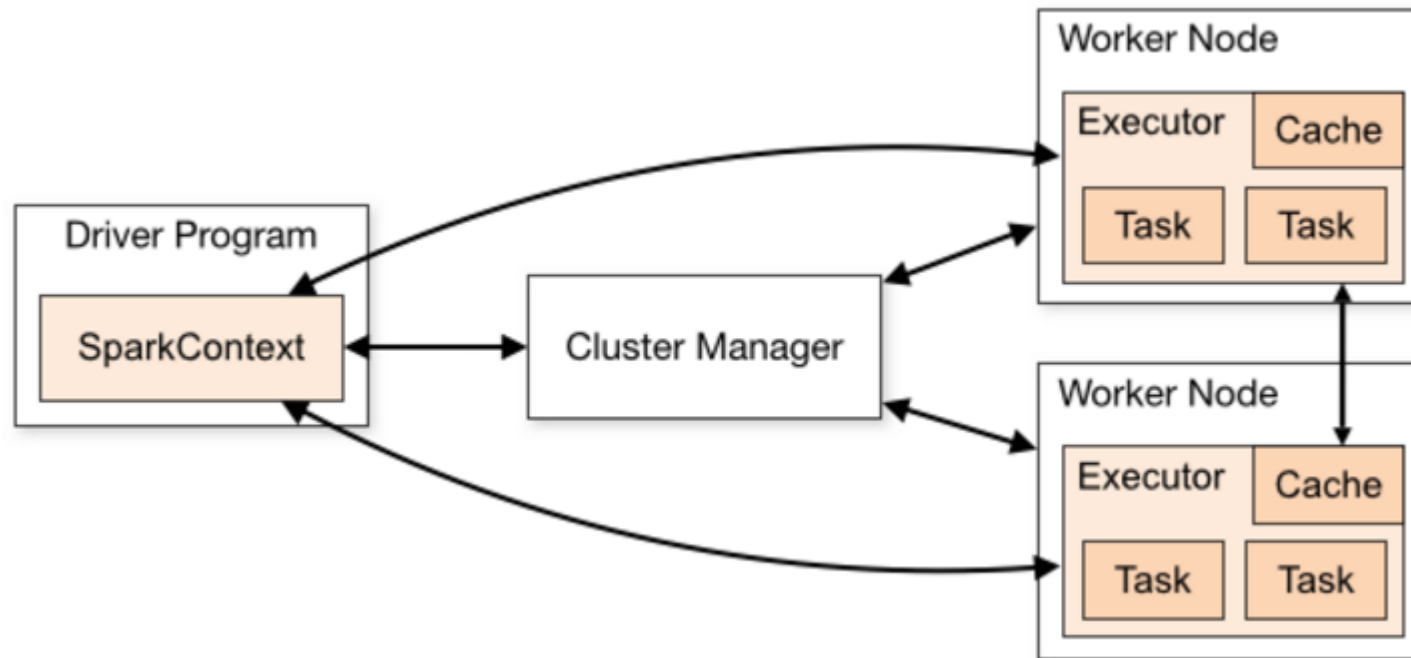


Part 3 – Using Spark in your Java Application

Spark Cluster



Spark cluster



Node tasks are mapped to cores on node

<http://spark.apache.org/>

The Driver Program

The Driver rules it all

- Connects to a cluster manager (Standalone/Mesos/Yarn) to allocate resources
- Acquires executors on clusters
- Sends app code to the executors
- Sends tasks for the executors to run

Spark

```
import org.apache.spark.api.java.JavaSparkContext;  
JavaSparkContext sc = new JavaSparkContext(sparkConfiguration);
```

Spark Configuration Object

```
SparkConf sparkConf =  
    new SparkConf().setAppName("MySparkApp").setMaster("spark://master:7077")
```

- If you run Spark locally on a *single* machine (as in our case!):

```
SparkConf sparkConf =  
    new SparkConf().setAppName("MySparkApp").setMaster("local[4]")
```

Number of cores Spark
should use on your
machine



- Further configuration options

```
new  
SparkConf().setAppName("MySparkApp").setMaster("local[4]").set("spark.executor.memory", "8g");
```

Maven dependency

```
groupId:    org.spark-project  
artifactId: spark-core_2.10  
version:    0.9.0
```

Update version
numbers, if necessary



Spark

```
import org.apache.spark.xyz;
```

```
...
```

```
import scala.Tuple2; //Always check that this is there!
```

```
public final class MySparkApplication {  
    public static void main(String[] args) {  
        SparkConf sparkConf = new  
            SparkConf().setAppName("MySparkApp").setMaster("local[4]").set("spark.executor.memory", "2g");  
        JavaSparkContext sc = new JavaSparkContext(sparkConf);  
        // Import data into RDDs...  
        // Compute results ...  
        sc.stop();  
        sc.close();  
    }  
}
```

Can use Spark in any class/method, not just main()


```
public final class WordCount {
    private static final Pattern SPACE = Pattern.compile(" ");
    public static void main(String[] args) throws Exception {
        SparkConf sparkConf = new SparkConf().setAppName("WordCount");
        JavaSparkContext ctx = new JavaSparkContext(sparkConf);

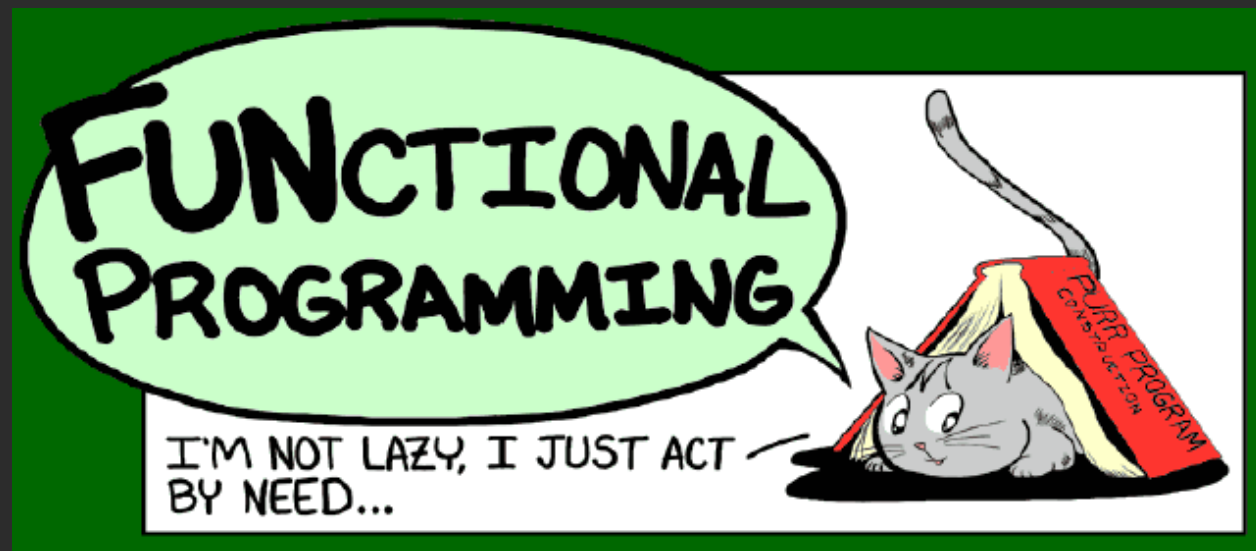
        JavaRDD<String> lines = ctx.textFile(pathToSomeTextFile, 1);
        JavaRDD<String> words = lines.flatMap((String s) -> Arrays.asList(SPACE.split(s))); // the words are our keys here
        JavaPairRDD<String, Integer> ones = words.mapToPair((String s) -> new Tuple2<String, Integer>(s, 1));
        JavaPairRDD<String, Integer> counts = ones.reduceByKey((Integer i1, Integer i2) -> i1 + i2);

        List<Tuple2<String, Integer>> output = counts.collect();
        for (Tuple2<?, ?> tuple : output) {
            System.out.println(tuple._1() + ": " + tuple._2());
        }
        ctx.stop();
        ctx.close(); // done automatically, but best practice to make close explicit (like for I/O objects)
    }
}
```

← Btw, why do we use flatMap here?

Remember...

- ... Spark allows arbitrary chains of operations
- ... RDDs are not restricted to key-value pairs
- ... Spark is Lazy
- ... Spark optimisation via `.cache` and `.parallelize`



Materialize when needed

Transformations (define a new RDD)	map filter sample <i>groupByKey</i> <i>reduceByKey</i> cogroup	flatMap union join cross mapValues ...
Actions (output a result)	collect reduce take fold	count saveAsTextFile saveAsHadoopFile ...

← Lazy (don't materialize)

← Materialize

End of Part 3 – Using Spark in your Java Application